
Topology-Aware Reinforcement Learning for Function Inlining

Łukasz Leszko (438580)

Abstract

Inlining-for-size is a fundamental compiler optimization technique that aims to reduce code size by replacing function calls with the body of the called function. However, finding the optimal inlining strategy is NP-hard. While manual heuristics are traditionally used, modern approaches formulate the problem as a Markov Decision Process and apply Reinforcement Learning (RL) to learn an inlining policy.

Recent work has demonstrated that RL can outperform these heuristics, yet it relies on simple, low-dimensional representations of the call graph, which limits its potential. In this paper, we propose an extension to the RL formulation that incorporates a richer call graph representation using modern graph embedding techniques (Graph Neural Networks).

1. Introduction

Function inlining is a fundamental compiler optimization technique where a function call from a caller to a callee is replaced by the body of the callee function. While this optimization can improve both execution speed and code size, this paper focuses on the latter. By itself, inlining strictly increases code size; however, when combined with other optimizations (e.g., constant folding and dead code elimination), it can lead to a significant reduction in binary size.

Compilers implement inlining by constructing a call graph, where nodes represent functions and directed edges represent function calls. Within this graph, the inlining problem is formulated by determining which edges to inline and in what order. Traditionally, compilers have relied on manually crafted heuristics to make these decisions. LLVM, for example, calculates a cost score for each call site and inlines only those with a score below a certain threshold.

Recently, MLGO (Trofin et al., 2021) proposed a Reinforcement Learning (RL) approach to learn an inlining policy. While they consider both Proximal Policy Optimization

TYPE	FEATURE
CALLER FEATURE	caller.basic.block.count
	caller.conditionally-executed.blocks
	caller.users
CALLEE FEATURE	callee.basic.block.count
	callee.conditionally-executed.blocks
	callee.users
CALL SITE FEATURE	callsite.height
	cost.estimate
	number.constant.params
CALL GRAPH FEATURE	edge.count
	node.count

Table 1. 11 features used in MLGO

(PPO) and evolution strategies, this paper focuses on the former. By iterating over the edges of the call graph, embedding each edge with a simple 11-feature vector, they trained an agent to make a binary decision on whether to inline an edge or not. This approach has shown significant improvements, achieving up to a 5.94% code size reduction over the default LLVM heuristic baseline, and has been successfully upstreamed into the LLVM compiler.

The primary limitation of MLGO is its reliance on a simple, low-dimensional representation of the call graph. Its vector embedding consists of only 11 features: 3 for the caller function, 3 for the callee function, 3 for the call site, and 2 global features (see Table 1). This simplistic representation fails to capture the complex, long-range topological structure of the call graph, which is crucial for the critic network to accurately estimate the value of the current state.

To address these limitations, this paper makes the following contributions:

- we develop `LLVMInline`, a custom reinforcement learning environment designed specifically for training inlining-for-size agents.
- we implement, evaluate and compare two graph embedding techniques: GraphSAGE and Graph Attention Networks (GAT).

2. Methodology

This paper builds upon the MLGO framework. we train a PPO agent on LLVM Intermediate Representation (IR)

The code is available at github.com/leszkolukasz/graph-mlgo.

to maximize code size reduction. However, our approach diverges from MLGO in two key aspects. First, we use a richer state representation. Second, we decouple the inlining process from the compiler: our agent applies inlining decisions externally, prior to the standard LLVM optimization passes, whereas MLGO integrates the agent directly into the LLVM source code.

2.1. Dataset

To train and evaluate our inlining agent, we utilize ComPile (Grossman et al., 2024), a 2 TB dataset of LLVM IR bitcode comprising programs written in languages such as C/C++, Rust, Swift, and Julia. From this dataset, we select a 1.5 GB subset containing approximately 33000 C-language IR modules with an average of 47.46 inlining opportunities per module.

Because the original dataset was generated across various LLVM releases, we first filter the modules to ensure compatibility with our local LLVM toolchain. Additionally, we discard overly complex modules to maintain training stability, restricting the call graphs to a maximum of 200 nodes, 500 edges, and 15,000 lines of code. Finally, we perform a validation step to ensure that exhaustively inlining all edges within a module completes without any compilation errors or memory exhaustion.

2.2. Environment

We implement `LLVMInline`, a custom reinforcement learning environment that conforms to the standard `gymnasium` interface.

For a given LLVM IR module, the creation of the call graph proceeds as follows:

First, the IR module is parsed to extract all functions (nodes) and call sites (edges). Next, the Strongly Connected Components (SCCs) are calculated, and only the edges between distinct SCCs are kept. This is done to ensure the acyclicity of the graph and to simplify the graph structure. If there is a recursive self-loop, it is retained but strictly limited to being unrolled only once.

Within this graph, the inlining operation is executed via a Foreign Function Interface (FFI) call to the underlying C++ code. This code uses LLVM libraries to perform the inlining operation and guarantee correctness of the updated module. While this could be achieved through a subprocess call, the FFI approach is significantly faster, as it avoids the overhead of spawning a new process for every single inlining decision.

During the inlining process, edges are visited using post-order traversal. This ensures that callee functions are inlined before their callers, allowing the inlining decisions to propa-

gate upwards through the call graph.

2.3. PPO

The inlining problem can be formulated as the following Markov Decision Process (MDP):

- **State space** $\mathcal{S}$: fixed-size embedding of the current call site, caller node, callee node and optionally surrounding graph structure.
- **Action space** $\mathcal{A}$: $\mathcal{A} = \{0, 1\}$, where 0 means "do not inline" and 1 means "inline".
- **State transition probability** $\mathcal{P}(s'|s, a)$: deterministic transition of the call graph under the inlining operation.
- **Reward** $\mathcal{R}(s, a)$: the change in code size after applying the inlining operation.

Defining the reward function requires isolation of the inlining effects. Intuitively, the step reward is formulated as:

$$\mathcal{R}(s, a) = \text{Size}(s) - \text{Size}(s') \quad (1)$$

where $\text{Size}(s)$ denotes the byte size of the `.text` section of the object file compiled from state s . Specifically, we utilize the LLVM `-Oz` pipeline with all inlining passes explicitly disabled.

2.4. Embedding

Graph Neural Networks (GNNs) typically operate in either a transductive or inductive setting. Transductive models, such as the original Graph Convolutional Networks (GCN) (Kipf & Welling, 2017), assume a fixed graph structure. In contrast, inductive problem works with dynamic graphs, where the structure can change over time which is the case in our setting. We use inductive architectures, specifically GraphSAGE (Hamilton et al., 2017) and Graph Attention Networks (GAT) (Veličković et al., 2018).

These methods operate through K message-passing layers, effectively capturing node's K -hop neighborhood.

2.4.1. GRAPHSAGE

GraphSAGE performs an iterative neighborhood aggregation (Algorithm 1). At each layer, a node updates its representation by aggregating the feature vectors of its local neighbors and applying a non-linear transformation. While the aggregation function can be implemented using mean, max, or LSTM pooling, we utilize max-pooling, which has been shown to demonstrate good performance (Hamilton et al., 2017).

Algorithm 1 GraphSAGE

Input: Graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$; input node features $\{x_v, \forall v \in \mathcal{V}\}$; edge types $\{e_{u,v}, \forall (u,v) \in \mathcal{E}\}$; depth K ; neighborhood function $\mathcal{N}(v)$

Parameters: light matrices W^k , input projection function InputProj , edge type encoder E_{enc} , layer normalizations LN_k , non-linearity σ , aggregator functions AGGREGATE_k

Output: Vector representations z_v for all target nodes

$h_v^0 \leftarrow \text{InputProj}(x_v), \forall v \in \mathcal{V}$

for $k = 1$ **to** K **do**

for each target node v **do**

$\tilde{h}_u^{k-1} \leftarrow h_u^{k-1} + E_{\text{enc}}(e_{u,v}), \forall u \in \mathcal{N}(v)$

$h_{\mathcal{N}(v)}^k \leftarrow \text{AGGREGATE}_k(\{\tilde{h}_u^{k-1}, \forall u \in \mathcal{N}(v)\})$

$h_v^k \leftarrow W^k \cdot \text{CONCAT}(h_v^{k-1}, h_{\mathcal{N}(v)}^k)$

if $k < K$ **then**

$h_v^k \leftarrow \sigma(\text{LN}_k(h_v^k))$

end if

$h_v^k \leftarrow h_v^k / \max(\|h_v^k\|_2, \epsilon)$

end for

end for

$z_v \leftarrow h_v^K, \forall v$

Note: Standard GraphSage considers undirected edges. In our case we need to differentiate between: self-loops, forward edges (caller $\rightarrow$ callee) and backward edges (callee $\rightarrow$ caller). E_{enc} represents a trainable embedding matrix for different types of call graph edges.

In the GraphSAGE algorithm information flows from nodes to their neighbours. There are however, modern alternatives that allow for more flexible information flow, e.g. if both edges and nodes have features, the information may flow first from nodes to edges and then from edges to nodes. The problem of interleaving node and edge updates is an open problem (Sanchez-Lengeling et al., 2021). This is however, beyond the scope of this paper.

2.4.2. GRAPH ATTENTION NETWORKS (GAT)

GAT is very similar to GraphSAGE, but instead of using a fixed pooling operator, GAT uses a self-attention mechanism (Veličković et al., 2018). Similar to our GraphSAGE implementation, we stack K attention layers, enabling each node to attend to its K -hop neighborhood, though unlike in text transformers, the positional encoding is replaced with the edge type encoding.

2.4.3. CALL GRAPH EMBEDDING

Given the edge to inline (u, v) the embedding is constructed as follows:

$$\text{embedding}(u, v) = \text{CONCAT}(h_G, h_u, h_v, h_{u \rightarrow v}, N_{\text{emb}}(u), N_{\text{emb}}(v)) \quad (2)$$

CATEGORY	FEATURE
TOPOLOGICAL	is_recursive
	in_degree
	in_degree_multiplicity
	out_degree
	out_degree_multiplicity
	node_height
IR METRICS	num_instructions
	num_blocks
	cond_blocks_ratio
	simple_instructions_ratio

Table 2. Node features in the LLVMInline environment. Includes more features than in the original MLGO.

where:

- h_G are global features: number of nodes, number of edges, number of unique edges.
- h_u and h_v are the node features as described in Table 2.
- $N_{\text{emb}}(u)$ and $N_{\text{emb}}(v)$ are the node embeddings generated by GraphSAGE or GAT.
- $h_{u \rightarrow v}$ are edge features: edge multiplicity, constant parameters ratio.

3. Evaluation

3.1. Baseline

The baseline is the default LLVM inlining heuristic.

3.2. Evaluation metrics

As we apply the inlining decisions externally, before the standard LLVM optimization passes, good evaluation metric should isolate inlining effects from other optimizations effects. For that reason, we use Optimized-Base Size Reduction (OSR) gain:

$$\text{Gain}_{\text{OSR}}(s) = \frac{\text{Size}_{\text{Oz-NoInline}}(s) - \text{Size}_{\text{Policy}}(s)}{\text{Size}_{\text{Oz-NoInline}}(s)} \times 100\% \quad (3)$$

where:

- $\text{Size}_{\text{Oz-NoInline}}(s)$ is the size of the .text section of the object file compiled from state s using the LLVM `-Oz` pipeline with all inlining passes explicitly disabled.
- $\text{Size}_{\text{Policy}}(s)$ is the size of the .text section of the object file compiled from state s :
 - after applying the inlining decisions from our policy and then compiling with the same `-Oz`

HYPERPARAMETER	VALUE
Learning Rate	1×10^{-4}
Discount Factor (γ)	0.99
GAE Parameter (λ)	0.95
Clipping Parameter (ϵ)	0.2
Entropy Coefficient	0.01
Rollout Length	1024

Table 3. Key PPO hyperparameters used for agent training.

- pipeline with inlining passes disabled if the policy is PPO-based, or
- after running the full `-Oz` pipeline with inlining passes enabled if the policy is heuristic-based.

For the sake of comparison with MLGO, we also report the Absolute Size Reduction (ASR) gain:

$$\text{Gain}_{\text{ASR}}(s) = \frac{\text{Size}_{\text{NoOpt}}(s) - \text{Size}_{\text{Policy}}(s)}{\text{Size}_{\text{NoOpt}}(s)} \times 100\% \quad (4)$$

where $\text{Size}_{\text{NoOpt}}(s)$ is the size of the .text section of the object file compiled from state s with `-O0` optimization level.

The exact metric scores can be found in Table 4 in the Appendix.

3.3. PPO

The baseline is a PPO agent where edge embeddings follow the equation 2 with node embeddings $N_{\text{emb}}(u)$ and $N_{\text{emb}}(v)$ removed. Agent’s configuration is described in Table 3. This agent and every subsequent configuration were trained on a total of 300’000 call sites which is around 6h of training on a single NVIDIA RTX 4060 GPU.

3.3.1. HIDDEN LAYER SIZE

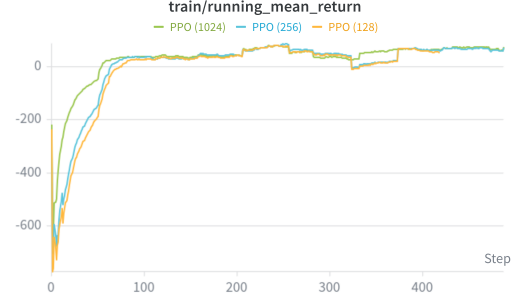
We evaluated the PPO agent’s performance across different hidden layer sizes (128, 256, and 1024) on reward calculated every 5 steps.

While the highest capacity model (1024) showed the fastest initial convergence in the mean return and the lowest early loss, it seems to have become overconfident as evidenced by lowest entropy and increased loss in later stages (see Figure 1).

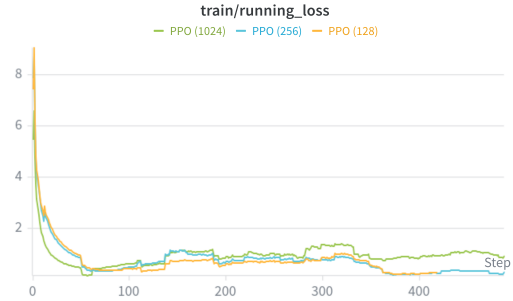
Smallest model (128) did not have enough capacity to learn a good policy resulting in the worst performance across all metrics and configurations.

Model with hidden layer size of 256 achieved the best overall performance nearly matching the baseline.

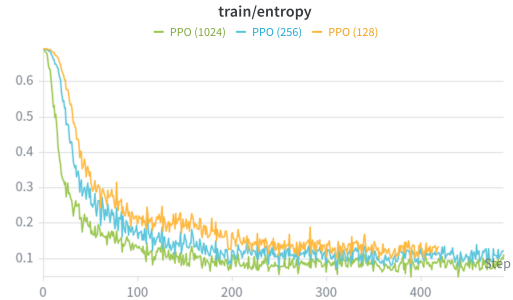
One important observation is that while the models were trained on around 500 different batches, the best validation performance was usually achieved much earlier, with severe degradation in later stages. For that reason, for all comparisons we used model weights from the best validation checkpoint.



(a) Running Mean Return



(b) Actor and Value Loss



(c) Policy Entropy

Figure 1. Training metrics for different hidden layer sizes.

3.3.2. REWARD SPARSITY

While dense reward is more informative, it requires compiling the module after every inlining decision, which is computationally expensive. For that reason, we compare the performance of the PPO agent trained with hidden size of 256 on sparse and dense rewards. Dense rewards were cal-

culated every 5 steps to reduce the computational overhead.

The sparse reward agent achieved better performance across all metrics and was better by around 0.3-0.4 percentage points compared to the baseline.

We believe that dense rewards caused model to focus too much on short-term gains. When inlining a callee into a caller, this may lead to a temporary decrease in code size, but may be a short-sighted decision if the caller is used later in the call graph. The difficulty in estimating the long-term effects of inlining are also shown in the increased value loss for the dense reward agent (see Figure 2).

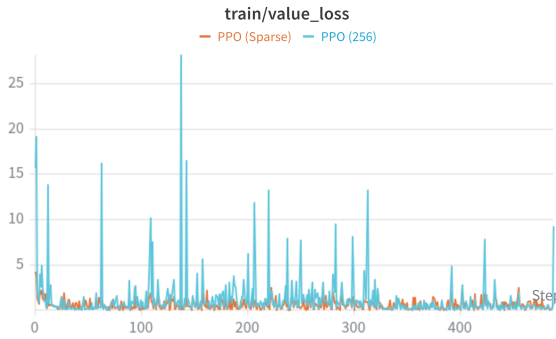


Figure 2. Training value loss for PPO agents trained with sparse and dense rewards.

3.4. Graph embedding

3.4.1. REWARD SPARSITY

While sparse rewards performed the best in the PPO agent without graph embedding, the situation is different when graph embedding is added to the state representation.

For the GAT agent, the dense reward configuration achieved better performance than the sparse reward configuration and seemed to have performed on par with the baseline.

For the GraphSAGE agent, the sparse reward configuration achieved better performance than the dense reward configuration and was better by around 0.1 percentage points compared to the baseline.

We assume that the increased parameter count caused by attention mechanism and feedforward layers in the GAT agent requires more informative reward signal to learn good embedding. This could be observed in the near zero evaluation reward for the sparse reward GAT agent which suggests that it defaulted to a simple policy of not inlining anything as perhaps embedding was too noisy to learn a better policy.

3.4.2. PRETRAINED WEIGHTS

Training the embedding models concurrently with the actor and critic networks can introduce significant instability into the training process. For instance, we observed that the gradients of the embedding models were, on average, twice the magnitude of those in the actor and critic networks. To mitigate this issue, we initialize the embedding models with pretrained weights and freeze them during the reinforcement learning phase.

We evaluate two pretraining strategies: an unsupervised approach using a contrastive loss, as proposed in the original GraphSAGE paper (Hamilton et al., 2017), and a supervised approach utilizing the weights obtained from the task described in the previous subsection.

Using the supervised pretrained weights yielded a marginal performance improvement of approximately 0.2 percentage points for the GAT model, whereas it notably degraded the performance of GraphSAGE.

Unsupervised pretraining degraded performance for both cases. The contrastive pretraining strategy aims to learn general-purpose embeddings by enforcing similar representations for nodes that are close in the graph. However, during our pretraining phase, the model weights failed to converge. We hypothesize that this behavior is primarily caused by the low dimensionality of the initial node features, which does not provide enough information to effectively categorize nodes.

3.4.3. RETAINING THE NODE’S ORIGINAL FEATURES

The embedding function 2 concatenates the node’s original features with the embedding generated by the GNN. GAT contains residual connections while GraphSAGE concatenates node embedding from phase $K - 1$ to embedding of neighborhood creating embedding for phase K which could suggest that the original features are not necessary. We evaluate the performance of the PPO agent with GAT and GraphSAGE embedding where the original node features are removed from the state representation (equation 2 is modified by removing h_u and h_v).

Removing the original node features resulted in performance degradation of around 0.6-0.85 percentage point for the GAT agent but was negligible for the GraphSAGE agent. We speculate that attention mechanism dilutes the node’s original features and relies more on the neighborhood information, while GraphSAGE’s architecture allows it to retain the original node features more effectively.

3.5. Discussion

Overall, we did not observe a substantial improvement over the baseline, with the sparse-reward PPO agent being the

best-performing model. Although this agent nominally outperformed the baseline on average, an analysis of the pairwise differences in Absolute Size Reduction (ASR) across the evaluation set reveals that the two approaches are nearly evenly matched. Specifically, the sparse agent outperformed the LLVM inlining heuristic in 47.65% of cases. However, its performance distribution exhibits heavier positive tail, resulting in a better average ASR and OSR gains (see Figure 3).

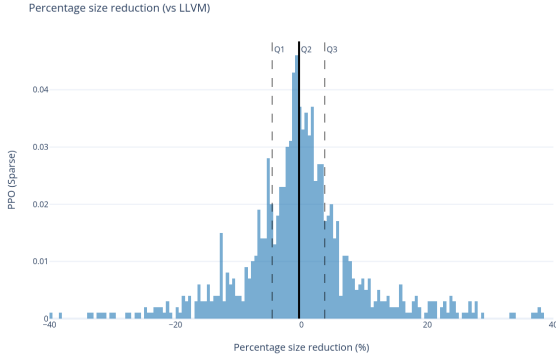


Figure 3. Histogram of the difference between PPO agent’s ASR gain and LLVM’s ASR gain across the evaluation set.

Training graph neural networks not only introduces more degrees of freedom but also adds more instability to the training process. More sophisticated training techniques or more extensive hyperparameter tuning are required to achieve better performance with graph embedding.

4. Future ideas

- **Unfrozen Pretraining:** While our current approach freezes the pretrained embedding model to prevent gradient instability, future work could explore concurrent fine-tuning.
- **Alternative RL Frameworks:** Although Proximal Policy Optimization (PPO) demonstrated strong performance, exploring other RL algorithms, such as Soft Actor-Critic (SAC), presents a promising direction, especially those that require less hyperparameter tuning and are more sample efficient.
- **Dilated Neighborhood Sampling:** Hardware memory constraints limited our GNN architecture to 8 message-passing layers. Perhaps increasing the number of layers would allow the model to capture more long-range dependencies in the call graph. An alternative would be adopting a dilated neighborhood sampling strategy. Instead of standard K -hop aggregation, the i -th layer could aggregate features from distant neighbors at a depth-proportional step, e.g., sampling nodes at

a distance of $\lfloor \frac{D}{K} \cdot i \rfloor$ for $i \in [1, K]$, where D is the maximum graph depth.

References

- Grossman, A., Paehler, L., Parasyris, K., Ben-Nun, T., Hegna, J., Moses, W., Diaz, J. M. M., Trofin, M., and Doerfert, J. Compile: A large ir dataset from production sources, 2024. URL <https://arxiv.org/abs/2309.15432>.
- Hamilton, W. L., Ying, R., and Leskovec, J. Inductive representation learning on large graphs. *CoRR*, abs/1706.02216, 2017. URL <http://arxiv.org/abs/1706.02216>.
- Kipf, T. N. and Welling, M. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017. URL <https://arxiv.org/abs/1609.02907>.
- Sanchez-Lengeling, B., Reif, E., Pearce, A., and Wiltchko, A. B. A gentle introduction to graph neural networks. *Distill*, 2021. doi: 10.23915/distill.00033. <https://distill.pub/2021/gnn-intro>.
- Trofin, M., Qian, Y., Brevdo, E., Lin, Z., Choromanski, K., and Li, D. Mlgo: a machine learning guided compiler optimizations framework, 2021. URL <https://arxiv.org/abs/2101.04808>.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. Graph attention networks, 2018. URL <https://arxiv.org/abs/1710.10903>.

A. Detailed metrics

Policy / Model	ASR (%)	OSR (%)
LLVM (Oz) Baseline	37.66	1.44
<i>No GNN</i>		
PPO (hidden=128)	26.89	-12.78
PPO (hidden=256)	37.58	1.38
PPO (hidden=1024)	36.71	0.11
PPO (Sparse Reward)	37.91	1.84
<i>GAT Architectures</i>		
PPO + GAT (Dense)	37.68	1.47
PPO + GAT (Sparse)	37.38	0.94
PPO + GAT (Contrastive)	37.02	0.55
PPO + GAT (Pretrained)	37.82	1.70
PPO + GAT (Pretrained, no Retain)	37.23	0.85
<i>GraphSAGE Architectures</i>		
PPO + GraphSAGE (Dense)	37.33	0.96
PPO + GraphSAGE (Sparse)	37.75	1.58
PPO + GraphSAGE (Contrastive)	36.81	0.15
PPO + GraphSAGE (Pretrained)	36.49	-0.22
PPO + GraphSAGE (Pretrained, no Retain)	36.66	-0.25

Table 4. Complete evaluation results for all PPO agents.

B. LLM usage

Gemini was successfully used for the following tasks:

- Understanding papers from the references.
- Writing LLVM passes and FFI code.
- Optimizing the code, refactoring JAX PPO code from Brax environment to work with our custom environment.
- Improving the report’s writing style and grammar and creating tables.
- Debugging the code and providing suggestions for improving the training stability.